

Compte rendu de TP

TD4 — Architecture MVC en PHP

Étudiant Délio Duchesne-Brajon
Bloc Bloc 2 — Développement
Outils PHP · PDO · MySQL · XAMPP
Date 17 septembre 2026

SOMMAIRE

1. Objectif du TP	2
2. Exercice 1 — Organisation du projet	2
2.1. Arborescence des dossiers	2
2.2. Le modèle ModelVoiture	2
2.3. Classe Model et configuration	3
3. Exercice 2 — La vue : liste des voitures	4
4. Exercice 3 — Le contrôleur	4
5. Exercice 4 — Le routeur	5
6. Exercice 5 — Action transmise dans l'URL	6
7. Exercice 6 — Détail d'une voiture	7
7.1. Vue de détail	7
7.2. Méthode read() du contrôleur	7
7.3. Liens depuis la liste	8
7.4. Gestion de l'immatriculation inconnue	8
8. Exercice 7 — Ajout d'une voiture	9
8.1. Formulaire de saisie	9
8.2. Contrôleur complet	10
8.3. Enregistrement dans le modèle	11
8.4. Résultat	12
9. Conclusion	13

1. Objectif du TP

Mettre en place une architecture MVC en PHP sur une base de voitures : séparer les données (modèle), l'affichage (vue) et le traitement des requêtes (contrôleur), puis introduire un routeur unique chargé de distribuer les actions.

Le projet est exécuté en local sous XAMPP, dans `htdocs/mvc`.

2. Exercice 1 — Organisation du projet

2.1. Arborescence des dossiers

Le projet est réorganisé en quatre dossiers : `config`, `controller`, `model` et `view`. Les fichiers de test écrits lors des TD précédents restent à la racine.

Nom	Modifié le	Type	Taille
config	17/09/2026 15:09	Dossier de fichiers	
controller	17/09/2026 15:09	Dossier de fichiers	
model view	17/09/2026 15:10	Dossier de fichiers	
view voiture	17/09/2026 15:10	Dossier de fichiers	
Conf.php	10/09/2026 12:52	Fichier source PHP	2 Ko
creerVoiture.php	17/09/2026 14:05	Fichier source PHP	1 Ko
formFindUtil.php	17/09/2026 15:02	Fichier source PHP	1 Ko
formulaireVoiture.html	17/09/2026 13:50	Chrome HTML Do...	2 Ko
lireTrajets.php	10/09/2026 13:44	Fichier source PHP	1 Ko
lireUtilisateurs.php	10/09/2026 13:44	Fichier source PHP	1 Ko
lireVoiture.php	17/09/2026 13:44	Fichier source PHP	1 Ko
Model.php	10/09/2026 12:58	Fichier source PHP	1 Ko
ModelVoiture.php	17/09/2026 15:12	Fichier source PHP	2 Ko
testConf.php	10/09/2026 12:35	Fichier source PHP	1 Ko
testFindUtil.php	17/09/2026 15:07	Fichier source PHP	1 Ko
testModel.php	10/09/2026 12:50	Fichier source PHP	1 Ko
Trajets.php	17/09/2026 15:00	Fichier source PHP	2 Ko
Utilisateurs.php	17/09/2026 14:59	Fichier source PHP	1 Ko

2.2. Le modèle ModelVoiture

La classe `ModelVoiture` regroupe l'accès aux données d'une voiture. La méthode statique `getAllVoitures()` exécute la requête, puis `setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture')` demande à PDO de convertir chaque ligne en objet de la classe plutôt qu'en tableau associatif.

```

Utilisateurs.php x  Trajets.php x  formFindUtil.php x  testFindUtil.php x  ModelVoiture.php x
3 > ModelVoiture.php
4 class ModelVoiture {
5
6     $req_prep = Model::$pdo->prepare($sql);
7     $values = array(
8         "nom_tag" => $immat,
9         //nomdutag => valeur, ...
10    );
11    // On donne les valeurs et on exécute la requête
12    $req_prep->execute($values);
13    // On récupère les résultats comme précédemment
14    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
15    $tab_voit = $req_prep->fetchAll();
16    // Attention, si il n'y a pas de résultats, on renvoie false
17    if (empty($tab_voit))
18    return false;
19    return $tab_voit[0];
20 }
21
22 public function save() {
23     $sql = "INSERT INTO voiture (immatriculation, marque, couleur)
24         VALUES (:immat, :marque, :couleur)";
25     $req_prep = Model::$pdo->prepare($sql);
26
27     $values = array(
28         "immat" => $this->immatriculation,
29         "marque" => $this->marque,
30         "couleur" => $this->couleur
31     );
32     $req_prep->execute($values);
33 }
34
35 public function __construct($m = NULL, $c = NULL, $i = NULL) {
36     if (!is_null($m) && !is_null($c) && !is_null($i)) {
37         $this->marque = $m;
38         $this->couleur = $c;
39         $this->immatriculation = $i;
40     }
41 }
42
43 //public function afficher() {
44 //    echo $this->marque . " " . $this->couleur . " (" . $this->immatriculation . ")<br>";
45 // }
46
47 public static function getAllVoitures() {
48     $rep = Model::$pdo->query("SELECT * FROM voiture");
49     $rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
50     return $rep->fetchAll();
51 }
52 }
53
54 ?>

```

```

// On récupère les résultats comme précédemment
$req_prep->setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture');
$tab_voit = $req_prep->fetchAll();
// Attention, si il n'y a pas de résultats, on renvoie false

```

2.3. Classe Model et configuration

Le dossier model contient Model.php, qui porte la connexion PDO partagée, et ModelVoiture.php. La connexion elle-même est isolée dans config/Conf.php, inclus par Model.php.

Ce PC > Disque local (C:) > xampp > htdocs > mvc > TD3 > model view				
Trier ▼ Afficher ▼ ...				
Nom	Modifié le	Type	Taille	
Model.php	10/09/2026 12:58	Fichier source PHP	1 Ko	
ModelVoiture.php	17/09/2026 15:12	Fichier source PHP	2 Ko	

Ce PC > Disque local (C:) > xampp > htdocs > mvc > TD3 > config				
Trier ▼ Afficher ▼ ...				
Nom	Modifié le	Type	Taille	
Conf.php	10/09/2026 12:52	Fichier source PHP	2 Ko	

```

1 <?php
2 require_once 'config/Conf.php';
3
4 class Model {
5     public static $pdo;

```

3. Exercice 2 — La vue : liste des voitures

La vue view/voiture/list.php ne contient aucune requête : elle parcourt le tableau d'objets reçu et affiche l'immatriculation de chaque voiture. C'est le principe de la séparation vue / modèle.

```

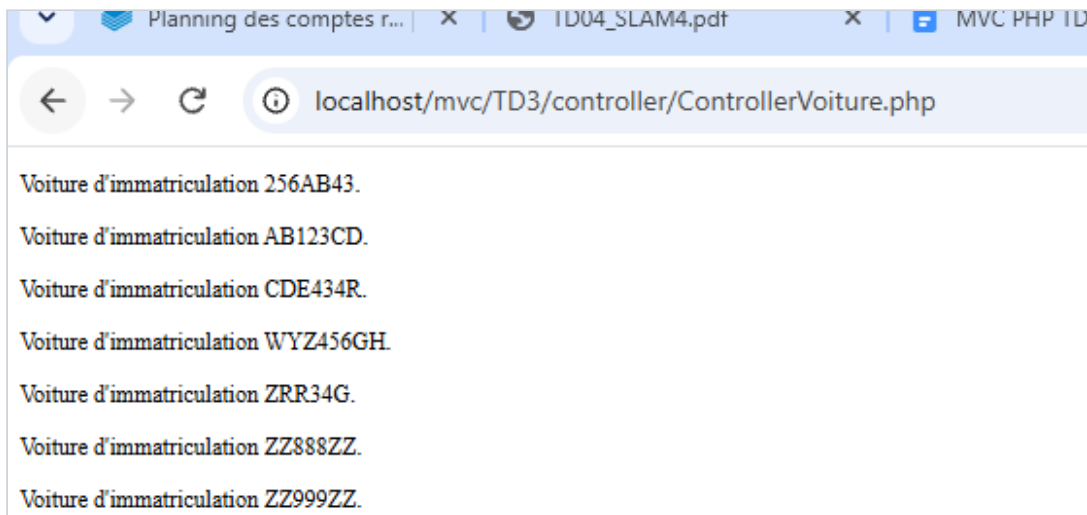
TD3 > view > voiture > list.php
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Liste des voitures</title>
6 </head>
7 <body>
8 <?php
9 foreach ($tab_v as $v)
10 echo '<p> Voiture d\'immatriculation ' . $v->getImmatriculation() . '</p>';
11 ?>
12 </body>
13 </html>

```

4. Exercice 3 — Le contrôleur

Le contrôleur fait le lien : il charge le modèle, récupère les données avec getAllVoitures(), puis inclut la vue. Appelé directement dans le navigateur, il affiche bien la liste des sept voitures de la base.

```
TD3 > controller > ControllerVoiture.php
1  <?php
2  require_once ('../model/ModelVoiture.php'); // chargement du modèle
3  $tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la BD
4  require ('../view/voiture/list.php'); //redirige vers la vue
5  ?>
6
```



5. Exercice 4 — Le routeur

Le contrôleur devient une classe, `ControllerVoiture`, dont le traitement est placé dans une méthode statique `readAll()`. Un nouveau fichier `routeur.php` devient le point d'entrée unique et appelle cette méthode.

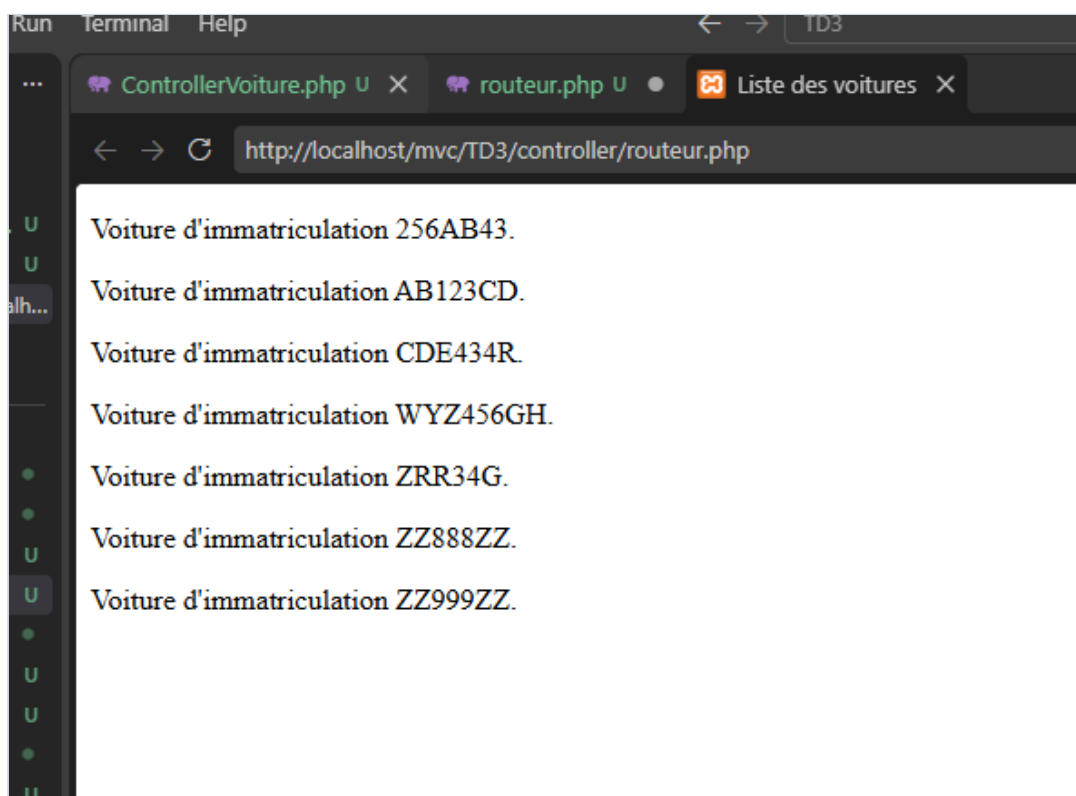
```
controller > routeur.php
1  <?php
2  require_once 'ControllerVoiture.php';
3  ControllerVoiture::readAll(); // Appel de la méthode statique $action de Controller
4  ?>
5
```

```

controller > ControllerVoiture.php
1  <?php
2  require_once ('../model/ModelVoiture.php'); // chargement du modèle
3  class ControllerVoiture {
4      public static function readAll() {
5          $stab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la BD
6          require ('../view/voiture/list.php'); // "redirige" vers la vue
7      }
8  }
9  ?>
10

```

Le résultat affiché est identique, mais l'URL passe désormais par le routeur.



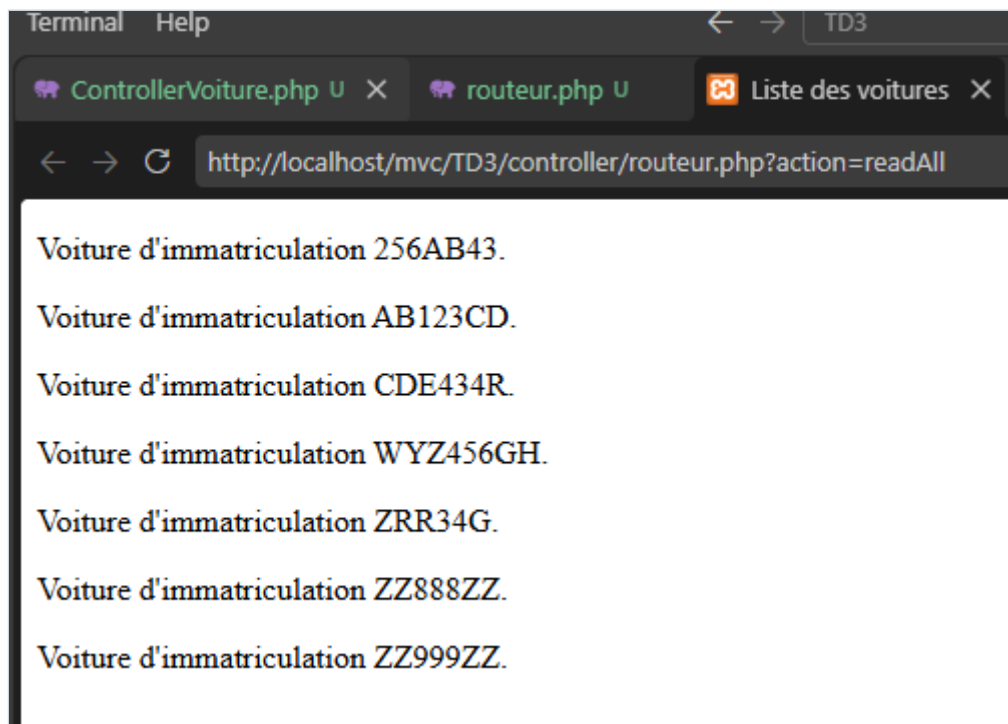
6. Exercice 5 — Action transmise dans l'URL

Le routeur ne fixe plus l'action en dur : il la lit dans l'URL avec `$_GET['action']` puis appelle dynamiquement `ControllerVoiture::$action()`. Une seule URL permet ainsi d'atteindre n'importe quelle action du contrôleur.

```

controller > routeur.php
1  <?php
2  require_once 'ControllerVoiture.php';
3  // On récupère l'action passée dans l'URL
4  $action = $_GET['action'];
5  ControllerVoiture::$action();
6  ?>
7

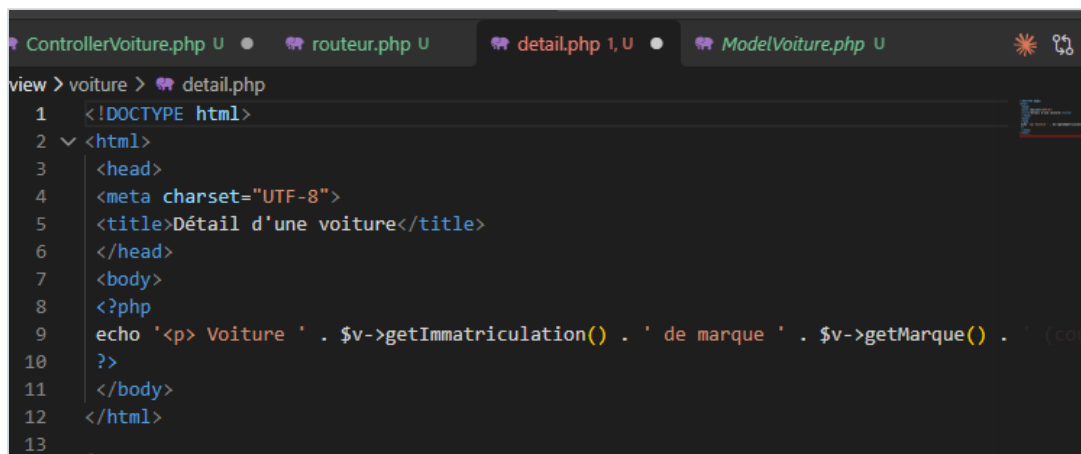
```



7. Exercice 6 — Détail d'une voiture

7.1. Vue de détail

Une seconde vue, `detail.php`, affiche l'immatriculation, la marque et la couleur d'une voiture donnée.



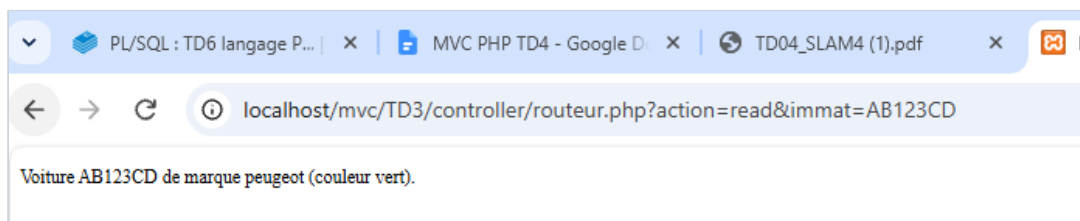
7.2. Méthode `read()` du contrôleur

La méthode `read()` récupère l'immatriculation dans l'URL et interroge le modèle avec `getVoitureByImmat()`, avant d'inclure la vue de détail.

```

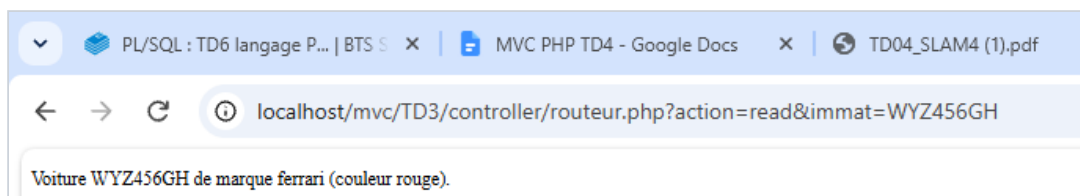
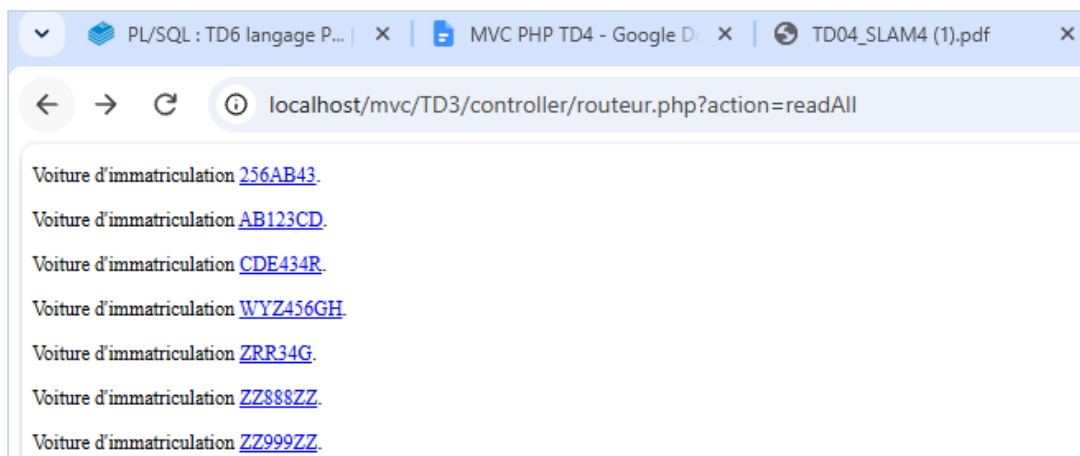
ControllerVoiture.php U • routeur.php U • detail.php 1, U • ModelVoiture.php U
controller > ControllerVoiture.php
1 <?php
2 require_once ('../model/ModelVoiture.php'); // chargement du modèle
3 class ControllerVoiture {
4     public static function readAll() {
5         $tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la BD
6         require ('../view/voiture/list.php'); // "redirige" vers la vue
7     }
8     public static function read() {
9         $immat = $_GET['immat'];
10        $v = ModelVoiture::getVoitureByImmat($immat);
11    }
12 }
13 ?>

```



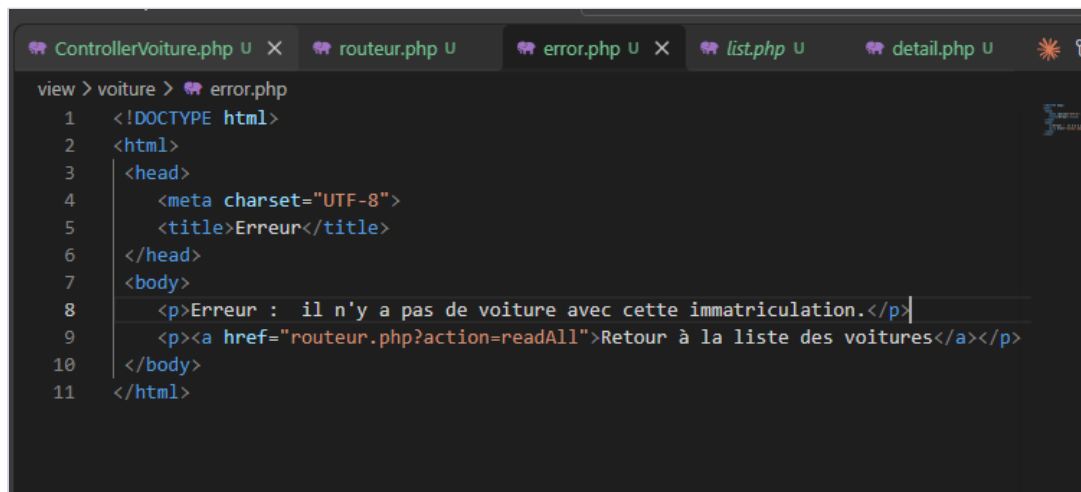
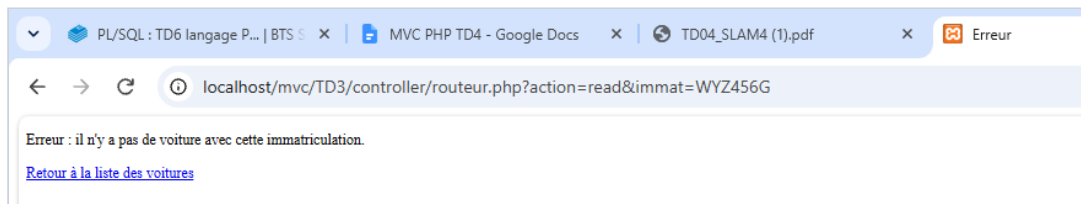
7.3. Liens depuis la liste

La vue liste est modifiée pour transformer chaque immatriculation en lien vers l'action read. La navigation entre la liste et le détail devient possible sans saisir l'URL à la main.



7.4. Gestion de l'immatriculation inconnue

Lorsque la recherche ne renvoie aucun résultat, le contrôleur teste `$v === false` et inclut une vue `error.php` plutôt que la vue de détail. Cette page affiche un message explicite et propose un retour vers la liste.



8. Exercice 7 — Ajout d'une voiture

8.1. Formulaire de saisie

La vue create.php contient un formulaire en method="post" dont l'action pointe vers routeur.php?action=created. Les champs marque et couleur sont marqués required.

```

view > voiture > create.php
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="utf-8" />
5          <title>Formulaire Voiture</title>
6      </head>
7      <body>
8          <form method="post" action="routeur.php?action=created">
9              <fieldset>
10                 <legend>Mon formulaire :</legend>
11                 <p>
12                     <label for="immat_id">Immatriculation</label> :
13                     <input type="text" placeholder="Ex : 256AB34" name="immatriculation">
14                 </p>
15                 <p>
16                     <label for="marque_id">Marque</label> :
17                     <input type="text" name="marque" id="marque_id" required/>
18                 </p>
19                 <p>
20                     <label for="couleur_id">Couleur</label> :
21                     <input type="text" name="couleur" id="couleur_id" required/>
22                 </p>
23                 <p>
24                     <input type="submit" value="Envoyer" />
25                 </p>
26             </fieldset>
27         </form>
28     </body>
29 </html>
30

```

Mon formulaire :

Immatriculation :

Marque :

Couleur :

8.2. Contrôleur complet

Deux actions sont ajoutées : `create()` affiche le formulaire, `created()` traite l'envoi. Cette dernière lit les champs dans `$_POST`, construit un objet `ModelVoiture` puis appelle `save()`.

```
terminal - help
ControllerVoiture.php U • routeur.php U error.php U create.php U detail.php U
controller > ControllerVoiture.php
1 <?php
2 require_once ('../model/ModelVoiture.php'); // chargement du modèle
3 class ControllerVoiture {
4     public static function readAll() {
5         $tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la BD
6         require ('../view/voiture/list.php'); //"redirige" vers la vue
7     }
8     public static function read(){
9         $immat = $_GET['immat'];
10        $v = ModelVoiture::getVoitureByImmat($immat);
11        if ($v == false)
12            require ('../view/voiture/error.php'); // immatriculation inconnue
13        else
14            require ('../view/voiture/detail.php');
15    }
16    public static function create() {
17        require ('../view/voiture/create.php'); // affiche le formulaire
18    }
19    public static function created() {
20        $immat = $_POST['immatriculation'];
21        $marque = $_POST['marque'];
22        $couleur = $_POST['couleur'];
23        $v = new ModelVoiture($immat, $marque, $couleur);
24        $v->save();
25        require ('../view/voiture/created.php');
26    }
27 }
28 ?>
29
```

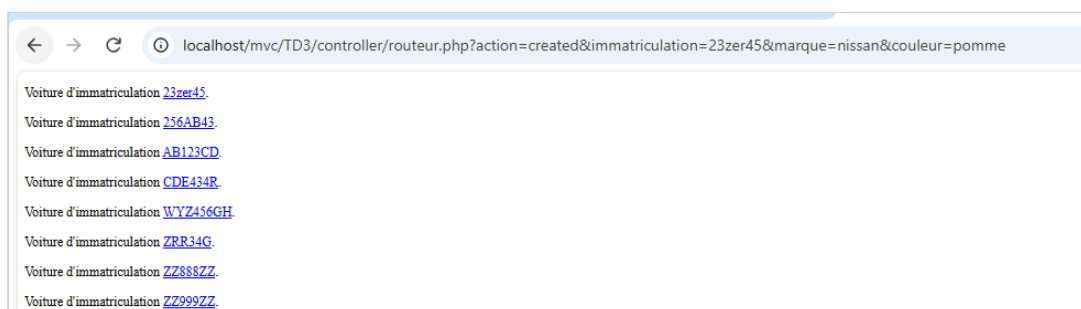
8.3. Enregistrement dans le modèle

La méthode `save()` effectue l'insertion au moyen d'une requête préparée : les valeurs saisies sont transmises par un tableau à `execute()`, et non concaténées dans la requête, ce qui protège contre l'injection SQL.

```
create.php U X ModelVoiture.php U ● routeur.php U error.php U detail.php U
model > ModelVoiture.php
25 }
28 }
29 public function getMarque() {
30 return $this->marque;
31 }
32 public function getCouleur() {
33 return $this->couleur;
34 }
35 public function save() {
36 $sql = "INSERT INTO voiture (immatriculation, marque, couleur)
37 | VALUES (:immat, :marque, :couleur)";
38 $req_prep = Model::$pdo->prepare($sql);
39
40 $values = array(
41 "immat" => $this->immatriculation,
42 "marque" => $this->marque,
43 "couleur" => $this->couleur
44 );
45 $req_prep->execute($values);
46 }
47
48 public function __construct($m = NULL, $c = NULL, $i = NULL) {
49 if (!is_null($m) && !is_null($c) && !is_null($i)) {
50 $this->marque = $m;
51 $this->couleur = $c;
52 $this->immatriculation = $i;
53 }
54 }
55
56
57 public static function getAllVoitures() {
58 $rep = Model::$pdo->query("SELECT * FROM voiture");
59 $rep->setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture');
60 return $rep->fetchAll();
61 }
62 }
63 ?>
```

8.4. Résultat

Après validation du formulaire, la nouvelle voiture apparaît dans la liste, qui compte désormais huit entrées.



9. Conclusion

Ce TD met en place une architecture MVC complète : le modèle porte les accès à la base, la vue se limite à l'affichage et le contrôleur orchestre les deux, le tout distribué par un routeur unique.

L'apport principal est le passage de l'action dans l'URL : en appelant dynamiquement la méthode correspondante, un seul point d'entrée suffit à gérer la consultation, le détail et l'ajout, sans multiplier les fichiers accessibles depuis le navigateur.